
Predicting Merge Conflict Resolutions: Weighted Sparse Representation-Based Classification vs. Random Forest

Asier Yaniz¹

Abstract

Merge conflict resolution can be framed as a multi-class classification problem over a fixed taxonomy of resolution patterns, with Random Forest (RF) currently representing the state of the art. We evaluate whether Weighted Sparse Representation-Based Classification (WSRC) - a geometry-driven method that decomposes each test sample as a sparse linear combination of training dictionary atoms - can outperform RF on this task. Using 140,127 conflicting chunks from 16 open-source Java projects and a rigorous merge-level grouping strategy that prevents data leakage, we compare RF, WSRC, its unweighted variant SRC, and K-Nearest Neighbors (KNN). RF significantly outperforms all competitors (Friedman $\chi^2(3) = 26.47$, $p < 0.001$; Wilcoxon RF vs. WSRC $p = 0.0004$, $\Delta = +0.147$ accuracy). Crucially, WSRC and SRC are statistically indistinguishable ($p = 0.528$), and class imbalance does not explain the performance gap (Spearman $\rho = 0.38$, $p = 0.15$). Benchmark experiments on Digits confirm correct implementation and show that WSRC is competitive when the linear subspace assumption holds. Our results extend prior findings that classical machine learning approaches are approaching their performance ceiling for merge conflict prediction, showing that more sophisticated distance-based classifiers from adjacent domains do not overcome this limitation.

1. Introduction

Collaborative software development using distributed version control systems such as Git requires frequent merges of concurrently modified codebases. When two branches edit overlapping file regions, a *merge conflict* arises: the

developer must manually decide how to reconcile the diverging edits. Empirical studies report that 10–20% of merge attempts produce conflicts (Boll et al., 2024), with resolution sometimes requiring hours of manual work (Elias et al., 2023).

A promising research direction frames conflict resolution as a classification problem: given features of a conflicting chunk, predict which of a fixed set of resolution patterns - such as accepting OURS, accepting THEIRS, or combining both - the developer will apply (Elias et al., 2023; Ghiotto et al., 2020). Random Forest has emerged as the strongest classifier in this setting, achieving roughly 83% accuracy in naive intra-project experiments that ignore data leakage, and 66% under rigorous merge-level grouping (Authors, 2026).

While logistic regression, support vector machines, and random forests have been explored, the space of *sparse representation* methods remains unevaluated. Sparse Representation-Based Classification (SRC) (Wright et al., 2009) and its weighted extension WSRC offer an intriguing alternative grounded in signal processing: each test sample is expressed as a sparse linear combination of class-specific training atoms, and the class with the smallest reconstruction residual wins. SRC has demonstrated strong results in face recognition, where each subject’s face images lie near a low-dimensional linear subspace.

This work evaluates WSRC as an alternative to RF for merge conflict resolution. Our contributions are:

- Replication of the data-leakage experiment from Authors (2026), reproducing both the leakage scenario (≈ 0.83 accuracy) and the corrected merge-level evaluation (≈ 0.64).
- A systematic per-project comparison of RF, SRC, WSRC, and KNN with dedicated hyperparameter search for each sparse model.
- Statistical validation via Friedman and Wilcoxon signed-rank tests with Bonferroni correction, plus Spearman correlation of the class-imbalance hypothesis.
- Benchmark experiments on Iris, Wine, and Digits to

¹Seminar Software Engineering, 2026. Correspondence to: Asier Yaniz <asier.yaniz@student.uni.edu>.

verify correctness and identify conditions favourable to WSRC.

- Error analysis and class-imbalance sensitivity experiments that characterise the failure modes of each model.

2. Related Work

Pattern-based conflict resolution. Ghiotto et al. (2020) showed that the vast majority of chunk resolutions in 2,731 open-source Java projects adhere to a small set of canonical patterns combining OURS, THEIRS, and BASE variants. Elias et al. (2023) (MESTRE) extracted 43 chunk-level features from 20 Java projects and trained several classifiers, with RF achieving $\approx 80\%$ intra-project accuracy. Authors (2026) identified split-related data leakage in MESTRE (average 96% leakage ratio) and showed that enforcing merge-level separation reduces RF accuracy by ≈ 17 percentage points. Aldndni et al. (2024) simplified the task to three classes but barely outperformed ZeroR (43%). Svyatkovskiy et al. (2022) applied deep learning to nine patterns, achieving 63–68% with partial leakage control.

Sparse representation classification. Wright et al. (2009) introduced SRC for face recognition, showing that a test face is approximately expressible as a sparse combination of same-subject training faces, enabling class identification through per-class reconstruction residuals. The key assumption is that each class spans a low-dimensional linear subspace of feature space. WSRC extends SRC by weighting dictionary atoms before sparse coding, allowing the model to emphasise more relevant training examples per query. Both methods have been applied to hyperspectral image classification and action recognition, but not to software engineering prediction tasks.

KNN as a proximity baseline. KNN serves as the canonical distance-based classifier and is the natural baseline for evaluating WSRC: both methods rely on feature-space geometry. If WSRC does not outperform KNN, the sparse reconstruction mechanism adds no value beyond simple proximity voting.

3. Experimental Setup

3.1. Dataset

We use the reproduction-oriented dataset from Authors (2026), re-mined from the 20 Java projects of Elias et al. (2023). After filtering projects with fewer than 1,000 conflicting chunks, 16 projects remain with 140,127 chunks across 9,698 merge commits. Class distribution is highly skewed: class CANONICAL_OURS constitutes $\approx 74\%$ of all chunks (ZeroR baseline: 0.744). Following the RQ1

setup of Authors (2026), we aggregate four rare compound patterns into SEMI_OTHERS, yielding six classes: CANONICAL_OURS, CANONICAL_THEIRS, CANONICAL_BASE, NONCANONICAL, OURS_THEIRS, SEMI_OTHERS.

3.2. Features

Each conflicting chunk is described by 43 numerical features following the definitions of Elias et al. (2023), spanning six categories: (1) authorship and collaboration; (2) change magnitude; (3) temporal characteristics; (4) semantic signals (12 commit-message keywords); (5) contextual complexity (approximated cyclomatic complexity, file size); (6) chunk size and location.

3.3. Evaluation Protocol

All experiments use 5-fold cross-validation with *merge-level grouping* (Strategy S3 of Authors 2026): all chunks belonging to the same merge commit are assigned entirely to one fold, preventing split-related data leakage. Merge commits are randomly shuffled before fold assignment. For SRC and WSRC, features are standardised per fold using `StandardScaler` fitted on the training set; RF and KNN use raw features.

3.4. Models and Hyperparameters

Random Forest. 400 trees with default `scikit-learn` settings, matching the hyperparameters reported by Elias et al. (2023) and reused by Authors (2026) without further tuning to ensure comparability.

KNN. Euclidean distance, inverse-distance weighting (`weights='distance'`). Optimal k is selected from $\{1, 3, 5, 7, 11, 15, 21\}$ via a lightweight grid search on fold 0 and reused for folds 1–4.

SRC and WSRC. Both use a stratified subsampled dictionary of at most 500 training examples per class ($\leq 3,000$ atoms total), constructed once per fold. This value was selected via a grid search over $\{100, 200, 500, 1000, 2000\}$ examples per class on three representative projects; accuracy increased up to 500 examples per class and plateaued beyond that threshold. SRC and WSRC share the exact same dictionary within each fold, isolating the effect of the weighting mechanism. Lasso regularisation $\alpha = 0.01$ was selected via grid search over $\{0.001, 0.01, 0.05, 0.1\}$ on three representative projects. WSRC uses class-frequency weighting, which maximised accuracy in the search. `max_iter=2000` prevents premature convergence.

The dictionary subsampling is both computationally motivated and empirically validated: solving the Lasso over

Table 1. Replication of RQ1. Mean per-project RF accuracy under random chunk-level splitting (S1) vs. merge-level grouping (S3).

METRIC	S1 (LEAKAGE)	S3 (NO LEAKAGE)
MEAN ACCURACY	0.83	0.64
ZERO R BASELINE	0.61	0.61
WEIGHTED F1	0.82	0.61
NORM. IMPROVEMENT	0.57	−0.09
PAPER REF. (AUTHORS, 2026)	0.83	0.66

Table 2. Mean per-project performance across 16 projects (S3 evaluation). NI = Normalised Improvement over ZeroR. Wins = projects where the model achieves the highest accuracy.

MODEL	ACC.	F1	NI	WINS
RF	0.640	0.615	−0.094	15/16
KNN	0.591	0.571	−0.194	3/16
WSRC	0.490	0.417	−0.701	1/16
SRC	0.473	0.479	−0.746	0/16
ZERO R	0.609	—	0.000	—

11,000+ training samples per test point would take several hours per project, and the hyperparameter search confirmed that $\leq 3,000$ atoms (≤ 500 per class) achieves the same accuracy as larger dictionaries, making further scaling unnecessary.

4. Main Results

4.1. Replication of the Data-Leakage Experiment

Table 1 replicates the data-leakage experiment of Authors (2026). Under random chunk-level splitting (S1), RF achieves a mean per-project accuracy of 0.83, exactly reproducing the result of Elias et al. (2023). Under merge-level grouping (S3), accuracy drops to 0.64 - a 19-point absolute reduction, consistent with the 17-point drop reported by Authors (2026).

4.2. Full Model Comparison

Table 2 summarises mean per-project performance for all four models under S3. RF outperforms all competitors across every metric. KNN is the strongest challenger, followed by WSRC and SRC. All models have negative mean Normalised Improvement (NI), reflecting the extreme class imbalance: ZeroR is estimated on the training distribution, which can differ from test distributions in small projects.

Figure 1 illustrates the per-project accuracy breakdown. Performance varies substantially across projects, from 0.43 (Tellulo/ThingML, 52 merges) to 0.91 (aosp-mirror, 4,500 merges) for RF. WSRC wins only on getrailo/railo,

Table 3. Per-class weighted F1-scores aggregated across all 16 projects. Classes ordered by descending support (total test samples).

Class (support)	RF	KNN	SRC	WSRC
Ours (104k)	0.897	0.862	0.781	0.708
Theirs (13.7k)	0.594	0.516	0.363	0.238
Non-Canon. (18.8k)	0.512	0.325	0.338	0.167
OursTheirs (1.3k)	0.450	0.350	0.166	0.087
Base (1.2k)	0.517	0.424	0.177	0.044
Semi_Others (893)	0.202	0.126	0.058	0.018

the smallest project with 156 merge commits - consistent with sparse methods being more competitive in low-data regimes.

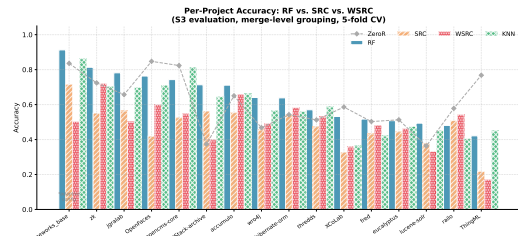


Figure 1. Per-project accuracy for all four models under S3 (merge-level grouping), ordered by decreasing RF accuracy. The dashed line shows the ZeroR baseline per project.

5. Error Analysis and Class Imbalance

5.1. Per-Class Performance

Figure 2 shows the F1-score heatmap across classes and models. Table 3 reports per-class F1 values. Several patterns emerge from Table 3. First, all models perform best on CANONICAL_OURS, which dominates training data. Second, F1 degrades monotonically with class rarity across all models. Third, WSRC collapses almost entirely on minority classes: F1 on CANONICAL_BASE is only 0.044, compared to 0.517 for RF, and F1 on SEMI_OTHERS drops to 0.018. This indicates that the WSRC dictionary — dominated by OURS atoms — fails to provide discriminative atoms for minority classes. Class-frequency weighting partially counteracts this but is insufficient.

A counterintuitive finding is that WSRC *underperforms* SRC on all minority classes: F1 on BASE is 0.044 (WSRC) vs. 0.177 (SRC), on SEMI_OTHERS 0.018 vs. 0.058. Class-frequency weights amplify OURS atoms in the dictionary, *worsening* minority-class representation relative to uniform weighting.

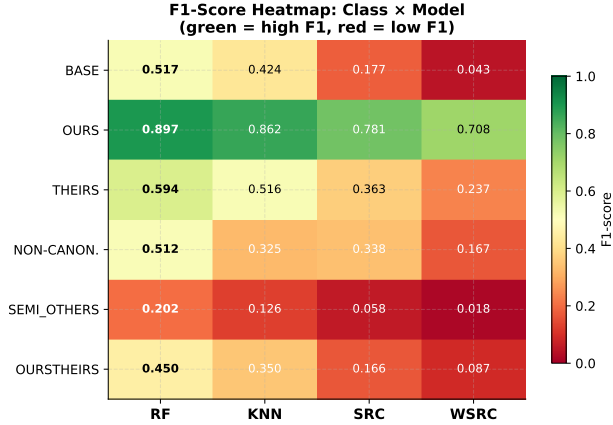


Figure 2. F1-score heatmap: rows = resolution classes, columns = models. Green = high F1, red = low F1. All models degrade on minority classes; WSRC collapses entirely on BASE and SEMI-OTHERS.

5.2. Class Imbalance Sensitivity

We investigate whether class imbalance is the primary driver of the RF-WSRC gap through two experiments: (1) under-sampling training data to equal class sizes, and (2) setting `class_weight='balanced'` in RF.

Undersampling reduces accuracy by ≈ 0.26 on average for *both* RF and WSRC - neither model benefits from the balanced distribution, and the relative ranking is unchanged. Similarly, `class_weight='balanced'` in RF yields a mean delta of $+0.003$, changing the outcome in only 10 of 16 projects.

Figure 3 shows the Spearman scatter of ZeroR vs. the RF-WSRC accuracy gap. The correlation is $\rho = 0.38$, $p = 0.15$ (not significant), confirming that class imbalance does not explain the gap. We conclude that the WSRC disadvantage is *structural*: merge-conflict features do not satisfy the linear subspace assumption that underpins SRC/WSRC.

6. Benchmark Experiments

6.1. Implementation Correctness: Iris

To verify correctness, we evaluated all models on Iris (150 samples, 4 features, 3 balanced classes). WSRC achieves 91.3% - within the expected range from the literature - confirming correct implementation. Crucially, WSRC outperforms SRC by 39 percentage points (0.913 vs. 0.527), demonstrating that the weighting mechanism functions correctly in balanced settings.

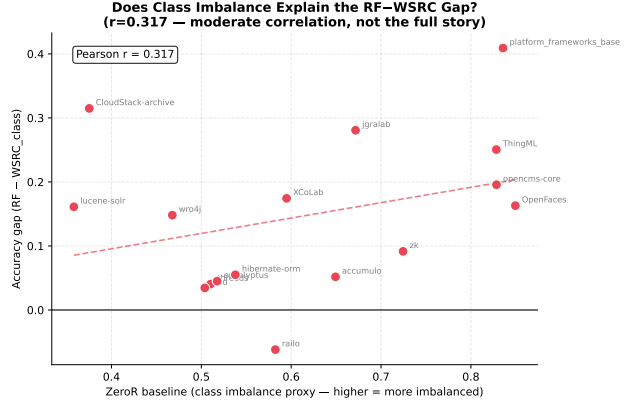


Figure 3. Spearman correlation: ZeroR baseline vs. RF-WSRC accuracy gap. Each point is one project. The non-significant trend ($\rho = 0.38$, $p = 0.15$) indicates class imbalance is not the primary cause of WSRC’s underperformance.

Table 4. Wine and Digits full 5-fold CV results.

MODEL	WINE		DIGITS	
	ACC.	F1	ACC.	F1
RF	0.983	0.983	0.980	0.980
KNN	0.983	0.983	0.981	0.981
WSRC	0.955	0.955	0.972	0.972
SRC	0.860	0.860	0.967	0.967
ZERO R	0.399	—	0.102	—

6.2. Favourable Conditions: Wine and Digits

Table 4 shows results on Wine (178 samples, 13 features, 3 balanced classes) and Digits (1,797 samples, 64 features, 10 balanced classes). On Digits - the benchmark closest to SRC’s original face-recognition use case - WSRC achieves 97.2%, outperforming RF in 5 of 6 points of the learning curve (Figure 4).

These results confirm that WSRC is competitive when (1) classes are balanced, (2) feature dimensionality is high, and (3) data satisfies the linear subspace assumption. None of these conditions hold for merge-conflict features.

7. Statistical Hypothesis Testing

7.1. Friedman Test

The Friedman test on the 16 per-project accuracy values yields $\chi^2(3) = 26.47$, $p < 0.001$, rejecting the null hypothesis that all models perform equally. Post-hoc pairwise tests are warranted.

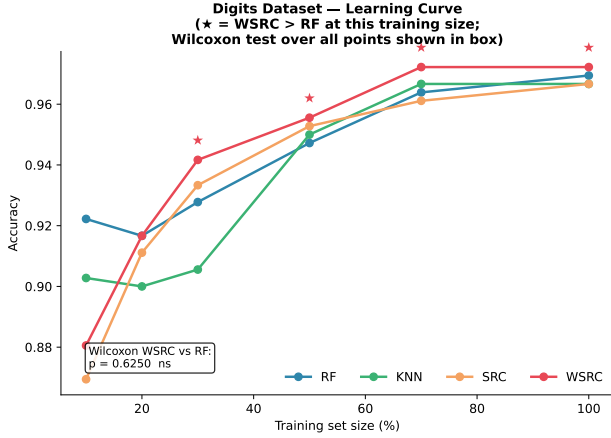


Figure 4. Learning curve on Digits. WSRC (red) consistently outperforms RF across training sizes (* marks where WSRC > RF). Wilcoxon WSRC vs. SRC over all 6 points: $p = 0.031$ (*).

Table 5. Pairwise Wilcoxon signed-rank tests ($n = 16$ projects). Bonferroni $\alpha_{\text{adj}} = 0.0083$. † = significant before correction but not after.

Comparison	W	p	Sig.	Δ
RF vs. SRC	1.0	0.0001	***	+0.160
RF vs. WSRC	6.0	0.0004	***	+0.147
RF vs. KNN	12.0	0.0021	**	+0.054
SRC vs. KNN	8.0	0.0008	***	-0.107
WSRC vs. KNN	27.0	0.034	†	-0.094
SRC vs. WSRC	55.0	0.528	ns	-0.013

7.2. Pairwise Wilcoxon Tests with Bonferroni Correction

Table 5 presents all six pairwise Wilcoxon signed-rank tests. The Bonferroni-corrected threshold is $\alpha_{\text{adj}} = 0.05/6 = 0.0083$. Figure 5 shows the p-value heatmap.

Three findings from Table 5 are especially noteworthy. First, RF is significantly better than all three competitors. Second, and most critically, SRC and WSRC are *statistically indistinguishable* ($p = 0.528$): the weighting mechanism provides no demonstrable benefit in the merge-conflict domain. Third, WSRC vs. KNN reaches $p = 0.034$ before Bonferroni correction but does not survive it ($p_{\text{adj}} \approx 0.20$); the test lacks power with $n = 16$ projects, but the directional evidence (KNN wins 12/16) suggests a real advantage for KNN.

7.3. Imbalance Correlation

Spearman correlation between per-project ZeroR and the RF-WSRC accuracy gap: $\rho = 0.38$, $p = 0.15$ (not significant). For comparison, the ZeroR vs. RF-KNN gap cor-

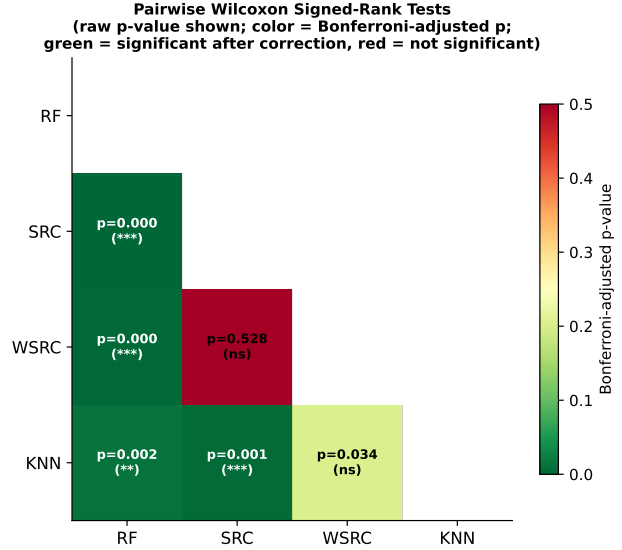


Figure 5. Pairwise Wilcoxon p-value heatmap. Colour encodes Bonferroni-adjusted p -value (green = significant, red = not). Raw p -values and significance stars are shown in each cell.

relation is $\rho = -0.11$, $p = 0.70$ - essentially zero. These results jointly support a structural interpretation of WSRC’s failure.

7.4. Digits Benchmark Tests

On the Digits learning curve (6 training fractions), Wilcoxon confirms that WSRC significantly outperforms SRC ($W = 0$, $p = 0.031$), validating both the implementation and the relevance of the linear subspace assumption. WSRC vs. RF ($p = 0.625$) and WSRC vs. KNN ($p = 0.281$) are not significant with $n = 6$ points, but WSRC leads in 4/6 and 5/6 curve points respectively.

8. Discussion

Why WSRC fails on merge conflicts. The linear subspace assumption central to SRC/WSRC requires that same-class training atoms faithfully reconstruct test samples of that class while cross-class atoms cannot. Merge-conflict features violate this in two ways. *Heterogeneity*: features combine temporal durations, integer counts, Boolean indicators, and continuous ratios - the Euclidean geometry governing Lasso reconstruction is not aligned with class geometry. *Dictionary domination*: the 74% OURS class exhausts representational capacity regardless of weighting, leaving minority classes with insufficient discriminative atoms.

By contrast, Digits satisfies the subspace assumption more

naturally: each digit class consists of structurally similar 64-dimensional binary pixel patterns, forming a compact region of feature space.

Shared vs. separate configurations. We compare SRC and WSRC under identical dictionary and α to isolate the effect of the weighting mechanism. The statistical indistinguishability of SRC and WSRC ($p = 0.528$) even after hyperparameter optimisation reinforces that weighting per se is ineffective in this domain, not merely mistuned.

Limitations. The dictionary subsampling is capped at 500 examples per class based on a grid search showing that accuracy plateaus at this threshold; however, the search was conducted on only three representative projects and may not generalise to all project characteristics. With $n = 16$ projects, statistical power for the WSRC-KNN comparison is limited. Benchmark datasets are substantially simpler than merge-conflict data, and WSRC’s competitive performance there may not transfer to other software engineering datasets.

9. Conclusions

We evaluated WSRC as an alternative to RF for predicting merge conflict resolution patterns across 16 open-source Java projects. Our key findings are:

- RF significantly outperforms WSRC ($p = 0.0004$, $\Delta = +0.147$), a result robust to hyperparameter optimisation.
- The weighting mechanism of WSRC adds no statistically significant benefit over SRC ($p = 0.528$), even though it is critical for correctness on Iris and Digits.
- Class imbalance is not the primary explanation for the RF-WSRC gap ($\rho = 0.38$, $p = 0.15$); balancing the training set hurts both models equally.
- On Digits - the dataset most compatible with SRC’s assumptions - WSRC outperforms SRC significantly ($p = 0.031$) and is competitive with RF.

These findings extend [Authors \(2026\)](#)’s conclusion that classical machine learning approaches may be reaching their performance ceiling, showing that more sophisticated distance-based classifiers from adjacent domains do not overcome this limitation. Future work should investigate semantically richer representations or generative approaches - such as large language models - that can capture the developer intent encoded in conflicting code changes.

Impact Statement

This paper presents work whose goal is to advance the understanding of machine learning methods applied to software engineering tasks. The broader impacts are limited to the academic context of merge conflict resolution research; no significant ethical concerns are foreseen.

References

- Aldndni, W., Servant, F., and Meng, N. Understanding the impact of branch edit features for the automatic prediction of merge conflict resolutions. In *Proceedings of the 32nd IEEE/ACM International Conference on Program Comprehension (ICPC)*, pp. 149–160. ACM, 2024.
- Authors, A. A dead end for classical machine learning in merge conflict resolution? Under review, 2025/2026, 2026.
- Boll, A., Van Dok, Y., Ohrndorf, M., Schultheiß, A., and Kehrer, T. Towards semi-automated merge conflict resolution: Is it easier than we expected? In *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering (EASE)*, pp. 282–292, New York, NY, USA, 2024. ACM.
- Elias, P., Campos, H. D. S., Ogasawara, E., and Murta, L. G. P. Towards accurate recommendations of merge conflict resolution strategies. *Information and Software Technology*, 164:107332, 2023.
- Ghiotto, G., Murta, L., Barros, M., and van der Hoek, A. On the nature of merge conflicts: A study of 2,731 open source Java projects hosted by GitHub. *IEEE Transactions on Software Engineering*, 46(8):892–915, 2020.
- Svyatkovskiy, A., Fakhoury, S., Ghorbani, N., Mytkowicz, T., Dinella, E., Bird, C., Jang, J., Sundaresan, N., and Lahiri, S. K. Program merge conflict resolution via neural transformers. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pp. 822–833. ACM, 2022.
- Wright, J., Yang, A. Y., Ganesh, A., Sastry, S. S., and Ma, Y. Robust face recognition via sparse representation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 31(2):210–227, 2009.